

Page Fault Handling

1 Core Idea

Definition

Page fault handling is the sequence of hardware and operating-system actions that loads a missing page into memory and restarts the faulting instruction.

A page fault is not just a disk read. It involves saving CPU state, validating the address, finding or freeing a frame, doing disk I/O, updating tables, and restarting execution.

Big Picture

trap → identify page → validate access → get frame → load page → restart instruction

2 Overview of the 10 Steps

Step	Main action
1	Hardware traps to the kernel and saves fault context.
2	Assembly routine saves registers and calls the OS.
3	OS identifies the faulting virtual page.
4	OS checks validity and protection, then finds or selects a frame.
5	If victim frame is dirty, schedule disk write and suspend the faulting process.
6	Read the needed page from disk into the clean frame.
7	Disk interrupt arrives; OS updates page tables and frame state.
8	Faulting instruction state is restored.
9	Faulting process is scheduled again.
10	Registers are reloaded and execution resumes in user space.

3 Step 1: Hardware Trap

The page fault begins in hardware.

Trap

The hardware traps to the kernel and saves the program counter. Many machines also save information about the current instruction in special CPU registers.

This saved information lets the OS later restart the instruction as if the page fault had not interrupted it.

4 Step 2: Save Volatile State

An assembly-language routine runs first.

Low-Level Save

The routine saves general registers and volatile CPU state so the operating system does not destroy the user process's execution context.

After saving state, the assembly routine calls the main operating-system page fault handler.

5 Step 3: Identify the Needed Page

The OS must determine which virtual address caused the fault.

Case	OS action
Hardware register gives address	Read the faulting virtual address directly.
No direct address register	Use the saved program counter, fetch the instruction, and decode it in software.

Hard Case

If hardware does not report the faulting virtual address, the OS may need to parse the faulting instruction to discover what memory reference caused the fault.

6 Step 4: Validate Address and Protection

Once the virtual address is known, the OS checks whether the access is legal.

Check	Result
Invalid virtual address	Send a signal or kill the process.
Protection violation	Send a signal or kill the process.
Valid page fault	Continue handling by finding a page frame.

If no free frame exists, the page replacement algorithm is run to choose a victim.

7 Step 5: Dirty Victim Handling

If the selected victim frame contains a dirty page, the page must be written back to disk.

Dirty Victim

A dirty page cannot simply be overwritten because disk does not yet contain its latest contents.

The OS schedules a disk write and marks the frame as busy. The faulting process is suspended, and another process may run while the disk operation completes.

Busy Frame

The selected frame is marked busy so another process does not try to use it before the write or read operation finishes.

8 Step 6: Read the Needed Page

Once the frame is clean, the OS locates the needed page on disk and schedules a disk read.

1. Find the disk address of the needed page.
2. Start disk I/O to read the page.
3. Keep the faulting process suspended during the read.
4. Run another ready process if one is available.

9 Step 7: Disk Interrupt and Table Update

When the disk finishes reading the page, it interrupts the CPU.

Page Arrived

The OS updates the page table so the faulting virtual page now maps to the page frame containing the loaded page.

The frame is marked as normal or available for ordinary use, not busy.

10 Step 8: Restore Faulting Instruction State

The faulting instruction must be retried from the beginning.

Instruction Restart

The OS backs up or resets the program counter so it points to the instruction that caused the page fault.

This is necessary because the memory reference failed before the instruction completed.

11 Steps 9 and 10: Resume the Process

After the page table has been fixed and the instruction state restored:

1. The faulting process is made runnable again.
2. The scheduler eventually chooses it.
3. The OS returns to the assembly routine that saved the registers.
4. That routine reloads registers and other saved state.
5. Control returns to user mode.

Result

The process continues as though the page fault never happened, except for the delay.

12 Clean Flow Summary

Phase	Key work
Trap and save	Preserve program counter, instruction state, and registers.
Diagnose	Determine the faulting address and virtual page.
Validate	Reject illegal addresses or protection violations.
Make room	Find free frame or evict a victim.
Disk I/O	Write dirty victim if needed; read missing page.
Repair mappings	Update page table and frame state.
Restart	Restore state and retry the faulting instruction.

13 Final Summary

One-Box Summary

A page fault traps from user mode into the kernel.
The OS finds the faulting page and checks legality.
It obtains a clean frame, reading/writing disk if needed.
It updates page tables and restarts the same instruction.
Correct restart makes the fault invisible except for time cost.